Python for Artificial Intelligence

Pandas II: Grouping, Merging & Exporting

Week 3, Day 3

Today's Agenda

- Hour 1: groupby() and aggregation, summarizing data by category
- Hour 2: Merging and concatenating DataFrames from separate sources
- Hour 3: Creating new columns, exporting results, and practice combining it all
- By the end: you can combine multiple tables and produce a summary ready to hand off

Hour 1

GroupBy & Aggregation

CONCEPT

Why Group Data?

- Many real questions are per-category: average score per city, total sales per region
- df.groupby('col') splits the DataFrame into one group per unique value in that column
- You then apply a function (mean, sum, count) to each group separately
- This split, apply, combine pattern is the core idea behind almost every summary you'll ever build

GroupBy Syntax

SYNTAX

```
df.groupby("city")["score"].mean()
```

PARAMETERS

`"city"` the column whose unique values define the groups

`["score"]` the column to summarize inside each group

`.mean()` the aggregation to apply; sum(), count(), max(), min() also work

YOU CAN

- ✓ Group by one column, or a list of columns for a more specific breakdown
- ✓ Chain a different aggregation: .sum(), .count(), .max()
- ✓ Apply several aggregations at once with .agg([...])

YOU CANNOT

- ✗ Group by a column and expect the original row order back unchanged
- ✗ Average a text column; aggregation needs numeric data

Returns: a Series (one column) or DataFrame (multiple columns), indexed by the group values

EXAMPLE

Average Score by City

groupby splits students into city groups, then mean() summarizes each one.

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Ali", "Sara", "Bilal", "Hina", "Omar"],
    "city": ["Karachi", "Lahore", "Karachi", "Islamabad", "Lahore"],
    "score": [85, 92, 78, 88, 95],
})

grouped = df.groupby("city")["score"].mean()
print(grouped)
# Islamabad 88.0, Karachi 81.5, Lahore 93.5
```

THINK & PRACTICE

Quick Check

Using a DataFrame orders with columns {region, amount} for 5 rows spread across "North" and "South", group by region and print the total amount per region using .sum().

ANSWER

Quick Check: Answer

```
import pandas as pd

orders = pd.DataFrame({
    "region": ["North", "South", "North", "South", "North"],
    "amount": [500, 300, 700, 200, 100],
})

totals = orders.groupby("region")["amount"].sum()
print(totals)
# North 1300, South 500
```

CONCEPT

Multiple Aggregations at Once

- .agg([...]) runs several summary functions on the same grouped column in one call
- .agg({...}) runs different summary functions on different columns, using a dictionary
- The result is a DataFrame with one row per group and one column per requested statistic
- This replaces writing three separate groupby lines with one clear call

EXAMPLE

Mean, Max and Count Together

One .agg() call produces a full mini-report, one row per city.

```
report = df.groupby("city")["score"].agg(["mean", "max", "count"])
print(report)
# each row: city, mean score, max score, number of students
```

THINK & PRACTICE

Quick Check

Using the orders DataFrame from before, group by region and use .agg() to get both the sum and the count of amount in one call. Print the result.

ANSWER

Quick Check: Answer

```
summary = orders.groupby("region")["amount"].agg(["sum", "count"])
print(summary)
# North: sum 1300, count 3 | South: sum 500, count 2
```

EXAMPLE

Aggregating Multiple Columns Differently

A dictionary lets each column get its own aggregation function in a single call.

```
import pandas as pd

df = pd.DataFrame({
    "city": ["Karachi", "Lahore", "Karachi", "Islamabad", "Lahore"],
    "score": [85, 92, 78, 88, 95],
    "age": [22, 21, 23, 20, 24],
})

summary = df.groupby("city").agg({"score": "mean", "age": "max"})
print(summary)
# each row: city, mean score, oldest student's age
```

THINK & PRACTICE

Quick Check

Using a DataFrame df with columns {department, salary, experience} for 6 employees across 3 departments (Sales, IT, HR), group by department and use a dictionary in .agg() to get the mean salary and the max experience for each department. Print the result.

ANSWER

Quick Check: Answer

```
import pandas as pd

df = pd.DataFrame({
    "department": ["Sales", "Sales", "IT", "IT", "HR", "HR"],
    "salary": [50000, 55000, 70000, 72000, 45000, 47000],
    "experience": [2, 3, 5, 6, 1, 2],
})

summary = df.groupby("department").agg({"salary": "mean", "experience": "max"})
print(summary)
# HR: mean 46000, max 2 | IT: mean 71000, max 6 | Sales: mean 52500, max 3
```

THINK & PRACTICE

Think & Practice (Harder)

Using the orders DataFrame, group by region and sum the amount, then sort the grouped result so the highest-total region appears first. Hint: groupby().sum() returns a Series, and Series have .sort_values() too.

ANSWER

Think & Practice (Harder): Answer

```
totals = orders.groupby("region")["amount"].sum()
ranked = totals.sort_values(ascending=False)
print(ranked)
# North 1300, South 500
```

Hour 2

Merging & Combining DataFrames

CONCEPT

Why Combine DataFrames?

- Real data is often split across tables: one file of students, another of their scores
- pd.merge() combines two DataFrames by matching values in a shared key column, like a SQL join
- pd.concat() stacks DataFrames on top of each other (or side by side), no matching key needed
- Choosing the right one depends on whether you're joining related data or stacking similar data

Merge Syntax

SYNTAX

```
pd.merge(students, scores, on="student_id", how="left")
```

PARAMETERS

`students, scores` the two DataFrames to combine

`on="student_id"` the shared column used to match rows between them, when both sides use the same name

`left_on=..., right_on=...` use instead of on= when the key column has a different name in each DataFrame

`how="left"` which rows to keep: "inner" (default), "left", "right", or "outer"

YOU CAN

✓ Match rows across two tables using a shared ID column

✓ Match on a key with different names using left_on="student_id", right_on="id"

✓ Keep every row from one side even when there's no match, with how="left" or "right"

✓ Keep every row from both sides with how="outer", filling gaps with NaN

Returns: a new, combined DataFrame

YOU CANNOT

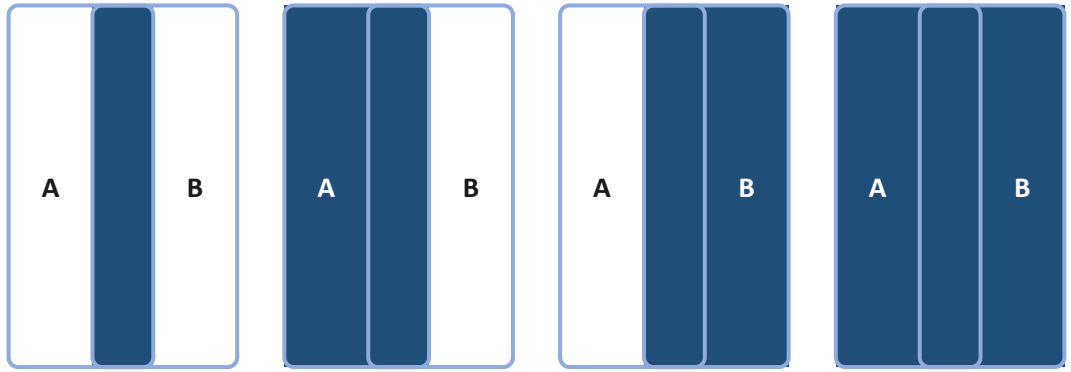
✗ Merge on a column that doesn't exist in both DataFrames without renaming first

✗ Expect an inner merge to keep rows that have no match on either side

CONCEPT

Merge Types, Visually

- Table A and Table B overlap where their key column has a match, the shaded area below
- Inner keeps only that shaded overlap: rows with a match on both sides
- Left keeps all of A plus the overlap; Right mirrors that, keeping all of B plus the overlap
- Outer keeps everything from both tables, matched or not, filling any gaps with NaN



INNER LEFT RIGHT OUTER

EXAMPLE

Inner vs. Left Merge

Hina (id 4) has no score, and id 5 has no matching student; each merge type handles that differently.

```
import pandas as pd

students = pd.DataFrame({
    "student_id": [1, 2, 3, 4],
    "name": ["Ali", "Sara", "Bilal", "Hina"],
})
scores = pd.DataFrame({
    "student_id": [1, 2, 3, 5],
    "score": [85, 92, 78, 99],
})

inner = pd.merge(students, scores, on="student_id")
print(inner)          # only ids 1, 2, 3 (matched on both sides)

left = pd.merge(students, scores, on="student_id", how="left")
print(left)           # all 4 students; Hina's score is NaN
```

EXAMPLE

Merging on Differently Named Columns

When the two tables don't use the same column name for the key, on= won't work, use left_on= and right_on= instead.

```
import pandas as pd

df1 = pd.DataFrame({
    "student_id": [1, 2, 3],
    "name": ["Ali", "Sara", "Bilal"],
})
df2 = pd.DataFrame({
    "id": [1, 2, 3],
    "score": [85, 92, 78],
})

merged = pd.merge(df1, df2, left_on="student_id", right_on="id")
print(merged)
# both key columns, student_id and id, appear in the result
```

THINK & PRACTICE

Quick Check

Create products (columns product_id, name for 3 products) and prices (columns product_id, price for the same 3 ids). Merge them on product_id with the default (inner) merge, and print the result.

ANSWER

Quick Check: Answer

```
import pandas as pd

products = pd.DataFrame({
    "product_id": [1, 2, 3],
    "name": ["Pen", "Bag", "Notebook"],
})
prices = pd.DataFrame({
    "product_id": [1, 2, 3],
    "price": [20, 900, 60],
})

merged = pd.merge(products, prices, on="product_id")
print(merged)
```

CONCEPT

Concatenating DataFrames

- pd.concat([df1, df2]) stacks two DataFrames that already share the same columns
- Use it when you have the same kind of data split across files, like sales for January and February
- ignore_index=True gives the combined result a fresh 0, 1, 2... index instead of repeating the originals
- Unlike merge, concat doesn't match on a key column, it just stacks the rows

EXAMPLE

Stacking Two Months of Sales

Both DataFrames have the same two columns, so they stack cleanly.

```
import pandas as pd

jan = pd.DataFrame({"day": [1, 2], "sales": [100, 150]})
feb = pd.DataFrame({"day": [1, 2], "sales": [120, 130]})

all_sales = pd.concat([jan, feb], ignore_index=True)
print(all_sales)
# 4 rows: 100, 150, 120, 130, with a clean 0-3 index
```

THINK & PRACTICE

Think & Practice (Harder)

Create storeA (columns item, qty for Pen=100, Bag=20) and storeB (columns item, qty for Pen=50, Notebook=30). Add a "store" column to each ("A" and "B"), concatenate them with ignore_index=True, then group the combined result by item and sum qty to get total stock per item across both stores.

ANSWER

Think & Practice (Harder): Answer

```
import pandas as pd

storeA = pd.DataFrame({"item": ["Pen", "Bag"], "qty": [100, 20]})
storeB = pd.DataFrame({"item": ["Pen", "Notebook"], "qty": [50, 30]})
storeA["store"] = "A"
storeB["store"] = "B"

combined = pd.concat([storeA, storeB], ignore_index=True)
totals = combined.groupby("item")["qty"].sum()

print(totals)
# Bag 20, Notebook 30, Pen 150
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

Hour 3

Feature Creation, Exporting & Practice

CONCEPT

Creating New Columns

- df['new_col'] = ... creates a brand new column from an expression on existing ones
- A comparison like df['score'] >= 80 creates a True/False (boolean) column directly
- df['col'].apply(function) runs a function on every value in that column, useful for custom logic
- This is called feature creation or feature engineering, and it's the first step toward machine learning

EXAMPLE

Adding a Pass/Fail and a Letter Grade

One new column comes from a comparison; the other comes from .apply() with a small function.

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Ali", "Sara", "Bilal"],
    "score": [85, 92, 78],
})

df["passed"] = df["score"] >= 80

def grade(score):
    if score >= 90: return "A"
    if score >= 80: return "B"
    return "C"

df["grade"] = df["score"].apply(grade)
print(df)
```

THINK & PRACTICE

Quick Check

Using a products DataFrame with columns {name, price, qty}, create a new column "total_value" equal to price multiplied by qty for each row. Print the result.

ANSWER

Quick Check: Answer

```
import pandas as pd

products = pd.DataFrame({
    "name": ["Pen", "Bag", "Notebook"],
    "price": [20, 900, 60],
    "qty": [100, 5, 40],
})

products["total_value"] = products["price"] * products["qty"]
print(products)
```

CONCEPT

Exporting Your Results

- df.to_csv('output.csv', index=False) saves a DataFrame as a CSV file
- df.to_excel('output.xlsx', index=False) saves it as an Excel file
- index=False leaves out the automatic 0, 1, 2... row numbers, which usually aren't useful in the file
- This is the last step of most data cleaning work: hand off a clean file, not just a variable in memory

WORKED EXAMPLE

Merge, Add a Column, Then Export

A realistic small pipeline: combine two sources, engineer a feature, and save the result.

```
import pandas as pd

students = pd.DataFrame({
    "student_id": [1, 2, 3],
    "name": ["Ali", "Sara", "Bilal"],
})
scores = pd.DataFrame({
    "student_id": [1, 2, 3],
    "score": [85, 92, 78],
})

merged = pd.merge(students, scores, on="student_id")
merged["passed"] = merged["score"] >= 80

merged.to_csv("results.csv", index=False)
```

THINK & PRACTICE

Practice

Using the orders DataFrame ({region, amount}), create a new column "amount_with_tax" equal to amount multiplied by 1.17. Then export the result to a CSV file named "orders_with_tax.csv" without the index column.

ANSWER

Practice: Answer

```
orders["amount_with_tax"] = orders["amount"] * 1.17
orders.to_csv("orders_with_tax.csv", index=False)
```

THINK & PRACTICE

Practice (Harder)

Using the orders DataFrame, group by region and sum amount. Use .reset_index() to turn the result back into a DataFrame with proper column names. Add a column "target_met" that is True where the summed amount is at least 1000. Export the final DataFrame to "region_summary.xlsx".

ANSWER

Practice (Harder): Answer

```
totals = orders.groupby("region")["amount"].sum()
summary = totals.reset_index()

summary["target_met"] = summary["amount"] >= 1000
print(summary)
# North 1300 True, South 500 False

summary.to_excel("region_summary.xlsx", index=False)
```

OPTIONAL

Extra Practice: The Superstore Dataset

- Download the "Sample Superstore" dataset from Kaggle (search "superstore dataset"), a real US retail chain's order history
- Columns include Category, Sub-Category, Region, Sales, Quantity, Discount, and Profit, one row per order line
- It's a realistic business dataset: the same kind of file a retail analyst gets asked to report on
- Use encoding="latin1" in pd.read_csv() if you get a UnicodeDecodeError loading it

OPTIONAL PROJECT CHECKPOINT

Practice: Superstore Profit Analysis

You're a data analyst for this retail chain. Using the Superstore dataset, your regional manager wants to know two things: which product Category earns the most total profit, and which Sub-Categories are actually losing money despite decent sales. Group by Category and Sub-Category to get total Sales and total Profit for each. Add a "profit_margin" column equal to Profit divided by Sales. Filter to the sub-categories with negative total profit and print them. Export the full summary to "category_profit_summary.csv".

ANSWER

Superstore Profit Analysis: Answer

```
import pandas as pd

df = pd.read_csv("Sample - Superstore.csv", encoding="latin1")

summary = df.groupby(["Category", "Sub-Category"]).agg(
    {"Sales": "sum", "Profit": "sum"}
)
summary["profit_margin"] = summary["Profit"] / summary["Sales"]

losing_money = summary[summary["Profit"] < 0]
print(losing_money)
# Tables and Bookcases typically show up here: high sales, negative profit

summary.to_csv("category_profit_summary.csv")
```

Home Practice: Interview-Style Questions

- In your own words, explain the split, apply, combine idea behind groupby()
- What is the difference between pd.merge() and pd.concat(), and when would you use each?
- What is the difference between an inner merge and a left merge?
- Why is .reset_index() often needed right after a groupby aggregation?
- What does index=False do when exporting to CSV, and why do you usually want it?
- Given two DataFrames of customers and their orders, write the steps to find total spend per customer

Key Takeaways & Coming Up

- groupby() splits data into categories so you can summarize each one with .mean(), .sum(), .agg()
- pd.merge() joins related tables on a shared key; pd.concat() stacks similar tables together
- New columns are created directly from expressions, or with .apply() for custom logic
- .to_csv() and .to_excel() turn a cleaned DataFrame into a shareable file
- Coming up next: Matplotlib and Seaborn, turning these DataFrames into charts